SmartDental – Guia Explicativo do Código, das Integrações e da Interface

Projeto II · Estevão Cicheleiro Gonçalves · Versão 1.0 · 28/10/2026

Este guia explica, de forma literal, o que cada arquivo, cada linha importante e cada palavra-chave do sistema SmartDental faz. Ele cobre: (1) o backend em Java com Spring Boot; (2) a ligação com o banco PostgreSQL; (3) a comunicação HTTP/JSON entre a interface e a API; (4) o arquivo SmartDental.bat; (5) a interface smartdental.ps1 em PowerShell.

1. Visão geral

1.1 As peças do sistema

Peça	Tecnologia / arquivo	O que faz
Banco de dados	PostgreSQL	Guarda pacientes, profissionais, agendamentos e atendimentos em tabelas.
Backend (API)	Java 17 + Spring Boot	Contém as regras de negócio. Recebe pedidos HTTP e conversa com o banco.
Interface	smartdental.ps1 (PowerShell)	Mostra um menu no terminal, envia pedidos HTTP à API e exibe as respostas em tabelas.
Inicializador	SmartDental.bat	Sobe o backend, espera a API ficar pronta e abre a interface.

1.2 O caminho de um pedido (exemplo: cadastrar paciente)

1. O usuário escolhe 'Cadastrar paciente' no menu e digita nome, CPF e telefone.
2. O PowerShell valida os dados e monta um texto JSON.
3. O PowerShell envia um pedido POST para http://localhost:8081/pacientes.
4. O PacienteController recebe o pedido e transforma o JSON em um objeto Paciente.
5. O PacienteService confere as regras (nome e CPF obrigatórios).
6. O PacienteRepository manda o JPA/Hibernate salvar o objeto.
7. O driver JDBC envia o comando SQL INSERT ao PostgreSQL, que grava a linha na tabela.
8. A resposta volta pelo mesmo caminho, com o paciente salvo (já com o id) em JSON.

9. O PowerShell lê o JSON e mostra: 'Paciente cadastrado com sucesso! ID: n'.

1.3 As camadas do backend

Camada	Pasta / pacote	Responsabilidade
Controller	com.smartdental.controller	Porta de entrada: define as rotas (URLs) e devolve as respostas HTTP.
Service	com.smartdental.service	Regras de negócio: validações, conflito de horário, inativar em vez de apagar.
Repository	com.smartdental.repository	Acesso ao banco: salvar, buscar, listar.
Entity	com.smartdental.entity	Molde de cada tabela: cada classe vira uma tabela, cada atributo vira uma coluna.

A regra de ouro é: o Controller só chama o Service, o Service só chama o Repository, e o Repository é quem fala com o banco. Cada camada conhece apenas a de baixo.

2. Glossário de conceitos

Termo	O que é
API	Conjunto de portas pelas quais um programa pode pedir serviços a outro. Aqui, o backend Java oferece a API e o PowerShell a consome.
REST	Estilo de API que usa URLs para identificar coisas (ex.: /pacientes) e métodos HTTP para dizer a ação (buscar, criar, alterar, apagar).
HTTP	Protocolo de comunicação usado por navegadores e programas para pedir e receber dados. Todo pedido tem método, endereço e, às vezes, corpo; toda resposta tem código de status e, às vezes, corpo.
Endpoint (rota)	Um endereço específico da API, como /pacientes/1.
GET	Método HTTP para ler dados. Não altera nada.
POST	Método HTTP para criar algo novo. Leva um corpo JSON.
PUT	Método HTTP para alterar algo que já existe.

Termo	O que é
DELETE	Método HTTP para remover. No SmartDental, em pacientes, ele apenas inativa.
JSON	Formato de texto para trocar dados. Exemplo: {"nome":"Maria","cpf":"12345678900"}. Chaves { } formam um objeto, colchetes [ ] formam uma lista.
Código de status	Número que resume a resposta. 200 = deu certo; 400 = pedido inválido (as mensagens de erro do SmartDental usam este); 404 = rota não existe; 500 = erro interno.
CRUD	Create, Read, Update, Delete: as quatro operações básicas (criar, ler, atualizar, apagar).
Porta	Número que identifica o 'portão' do computador onde um programa escuta. A API escuta na 8081; o PostgreSQL, na 5432.
localhost	Nome que significa 'este mesmo computador'.
Framework	Conjunto de código pronto que dá a estrutura do programa. O Spring Boot é o framework do backend.
Spring Boot	Framework Java que monta sozinho um servidor web e conecta as peças com pouca configuração.
Maven	Ferramenta que baixa as bibliotecas do projeto, compila o código e roda a aplicação. Lê o arquivo pom.xml.
Dependência	Biblioteca externa de que o projeto precisa.
JPA	Especificação Java que permite tratar linhas de tabelas como objetos Java.
Hibernate	Implementação do JPA usada pelo Spring Boot. Gera o SQL por baixo dos panos.
ORM	Técnica de mapear objetos para tabelas (Object-Relational Mapping). JPA e Hibernate fazem isso.
JDBC	Padrão Java para conversar com bancos de dados. O driver do PostgreSQL implementa este padrão.
SGBD	Sistema Gerenciador de Banco de Dados. Aqui, o PostgreSQL.

Termo	O que é
Chave primária (PK)	Coluna que identifica cada linha de forma única. Aqui, sempre o campo id.
Bean	Objeto criado e controlado pelo Spring (as classes marcadas com @Service, @RestController etc.).
Injeção de dependências	O Spring entrega prontos os objetos de que uma classe precisa, em vez de ela criá-los. É o que o @Autowired faz.
Annotation (@)	Etiqueta colocada sobre uma classe, campo ou método para dar uma instrução ao Spring/JPA.
Tomcat embutido	Servidor web que já vem dentro do Spring Boot; é ele que recebe os pedidos na porta 8081.
Jackson	Biblioteca que converte objeto Java em JSON e JSON em objeto Java automaticamente.
UTF-8	Codificação de texto que representa corretamente acentos e símbolos.
ISO-8601	Padrão de data e hora em texto, como 2026-11-10T14:00:00 (o T separa a data da hora).

3. Backend: arquivos de configuração

3.1 pom.xml

O pom.xml (Project Object Model) é a 'receita' do projeto para o Maven. É escrito em XML, em que cada informação fica entre uma etiqueta de abertura e uma de fechamento.

Trecho	O que faz
<?xml version="1.0" encoding="UTF-8"?>	Declara que o arquivo é XML, versão 1.0, em UTF-8.
<project xmlns=... xsi:schemaLocation=...>	Etiqueta raiz que envolve tudo. Os atributos xmlns indicam o 'dicionário' (esquema) que define as etiquetas válidas; servem só para validação.
<modelVersion>4.0.0</modelVersion>	Versão do formato do POM. Sempre 4.0.0 no Maven atual.

Trecho	O que faz
<parent>	Faz o projeto herdar configurações de outro POM.
spring-boot-starter-parent 3.2.5	O POM pai: define as versões compatíveis de todas as bibliotecas Spring, então não é preciso informar versão em cada dependência. 3.2.5 é a versão do Spring Boot.
<relativePath/>	Vazio: manda o Maven buscar o pai na internet/repositório, não numa pasta local.
<groupId>com.smartdental</groupId>	Identificador do grupo/organização do projeto.
<artifactId>smartdental</artifactId>	Nome do projeto/arquivo gerado.
<version>0.0.1</version>	Versão do projeto.
<name>smartdental</name>	Nome amigável.
<properties> <java.version>17</java.version>	Define que o código usa Java 17.
<dependencies>	Lista as bibliotecas que o projeto usa.
spring-boot-starter-web	Traz o Spring MVC (rotas e controllers), o Tomcat embutido e o Jackson (JSON). É o que permite o programa ser uma API web.
spring-boot-starter-data-jpa	Traz o JPA, o Hibernate, o Spring Data (os repositories prontos) e o pool de conexões HikariCP.
postgresql (scope runtime)	Driver JDBC do PostgreSQL. O escopo runtime significa que só é necessário ao executar, não para compilar.
<build><plugins>	Configura a etapa de construção.
spring-boot-maven-plugin	Plugin que permite rodar com mvn spring-boot:run e empacotar a aplicação num único arquivo executável.

3.2 application.properties

Arquivo de configurações do Spring, escrito como chave=valor. Fica em src/main/resources.

Linha	O que faz
# texto	Linha de comentário; é ignorada.
spring.datasource.url=jdbc:postgresql://localhost:5432/smartdental	Endereço do banco. jdbc:postgresql = protocolo/driver; localhost = o banco está neste computador; 5432 = porta do PostgreSQL; smartdental = nome do banco de dados.
spring.datasource.username=postgres	Usuário usado para entrar no banco.
spring.datasource.password=...	Senha desse usuário.
spring.jpa.hibernate.ddl-auto=update	Diz ao Hibernate o que fazer com as tabelas ao iniciar. 'update' cria as tabelas que faltam e acrescenta colunas novas, sem apagar dados existentes. Outros valores: none (não mexe), validate (só confere), create (apaga e recria tudo) e create-drop (apaga ao encerrar).
spring.jpa.hibernate.show-sql / spring.jpa.show-sql=true	Mostra no console cada comando SQL executado. Útil para estudar e depurar.
server.port=...	Porta em que a API escuta. No projeto entregue era 8080; no seu computador está 8081, e é a porta que o .bat e o .ps1 usam.

3.3 SmartdentalApplication.java

É o ponto de partida do backend.

Trecho	O que faz
package com.smartdental;	Declara a pasta lógica (pacote) em que a classe está.
import org.springframework...;	Traz classes de outras bibliotecas para poderem ser usadas aqui.

Trecho	O que faz
@SpringBootApplication	Etiqueta que junta três instruções: @Configuration (esta classe contém configurações), @EnableAutoConfiguration (o Spring configura sozinho o que encontrar, como servidor web e banco) e @ComponentScan (procure classes marcadas com @Service, @RestController etc. neste pacote e nos de dentro).
public class SmartdentalApplication	Declara a classe. 'public' = visível para todos; 'class' = define uma classe.
public static void main(String[] args)	O método que o Java executa primeiro. 'static' = pode ser chamado sem criar objeto; 'void' = não devolve valor; 'String[] args' = lista de argumentos recebidos na linha de comando.
SpringApplication.run(SmartdentalApplication.class, args);	Liga o Spring Boot: cria os beans, conecta no banco, sobe o Tomcat e passa a escutar a porta.
System.out.println("SmartDental iniciou!");	Escreve esse texto no console.